Lecture 8 — Geometry II: Rotating Calipers, Half-Plane Intersection, Closest Pair, Angular Sweep

> National Championship Prep · Prerequisites: Year week 31 (vectors, cross product, orientation, convex hull), Regional 08 (point-in-polygon, line intersection, precision), Year week 13 (ordered sets) · Estimated time: 2 × 75 min lecture + 3 h practice

1. Overview

Regional geometry is mostly "get the primitives right": orientation tests, segment intersection, polygon area, convex hull. National sets add one more layer: an $O(n^2)$ brute force over pairs of points or pairs of edges is obvious, and the problem is to find the $O(n \log n)$ structure. Four techniques cover a large share of these problems:

- **Rotating calipers** — after building the convex hull, sweep a pair of parallel support lines around it. Antipodal pairs are enumerated in $O(n)$, giving the diameter, width, minimum bounding rectangle, and more.
- **Half-plane intersection (HPI)** — the intersection of n half-planes is a convex polygon computable in $O(n \log n)$ by sorting by angle and maintaining a deque. Combined with binary search on the answer it solves "largest circle / farthest point inside" problems.
- **Closest pair of points** — a sweep line with a y-ordered window, or divide and conquer, in $O(n \log n)$, justified by a packing argument.
- **Angular sweep** — sort directions around a center exactly (no `atan2`), then use two pointers or events as a ray rotates. Many counting problems ("how many triangles contain the point", "max points in a circle of radius r") become $O(n \log n)$ per center.

Throughout, we follow a **precision policy**: stay in integers (`long long`, `__int128`) for predicates whenever the input is integral, compare squared distances, and introduce `long double` and ε only where a constructed point (an intersection) is compared.

2. Rotating calipers

2.1 Antipodal pairs

Let H be a convex polygon in counter-clockwise order with no three collinear vertices. Vertices u and v are **antipodal** if there are two parallel lines, one through u and one through v, with H between them. The farthest pair of points of any set is attained by two hull vertices (the distance function is convex, so its maximum over a polygon is at a vertex), and moreover by an antipodal pair: draw the lines through the farthest pair perpendicular to the segment joining them; every other point lies between them, otherwise it would be farther away.

For each edge (i, i+1), let j be the vertex farthest from the line through the edge (the vertex maximizing the triangle area `cross(h[i+1]−h[i], h[j]−h[i])`). As i moves counter-clockwise, j also moves counter-clockwise, never backwards — the support line parallel to edge i rotates monotonically. So a single pointer j that advances while the area increases visits O(n) positions in total. The area increases from j to j+1 exactly when the edge (j, j+1) still points "against" edge i: `cross(h[i+1]−h[i], h[j+1]−h[j]) > 0`.

Each pair (i, j) and (i+1, j) found this way is antipodal, and every antipodal pair appears, so the diameter is the maximum over them. **Width** (the minimum distance between two parallel support lines) is always attained with one line flush against a hull edge, so it is `min_i area(i, i+1, j) / |edge i|` over the same loop.

2.2 More calipers

- **Minimum-area (or minimum-perimeter) enclosing rectangle.** Freeman and Shapira (1975) proved the optimal rectangle has a side collinear with a hull edge. For each edge keep three pointers: the farthest vertex along the edge direction, the farthest against it, and the farthest from the edge. All three move monotonically: O(n).
- **Max/min distance between two convex polygons** — calipers on both polygons simultaneously (or the Minkowski sum of one with the reflection of the other; World Finals Lecture 6).
- **Largest-area triangle on n points** — the vertices lie on the hull; for a fixed first vertex the other two can be moved with two monotone pointers, giving $O(n^2)$ (an O(n) algorithm exists but is subtle).

2.3 Preconditions

The calipers loop assumes a strictly convex polygon. Remove duplicate points and collinear points from the hull (the monotone chain with `<= 0` pops does this), and special-case hulls of 1 or 2 points (all input points collinear). Feeding a hull that contains collinear points can make j stall on a zero-area step and miss the true antipode.

3. Half-plane intersection

A half-plane is represented by a directed line `p → p + d`; it contains the points on its **left** (`cross(d, r − p) ≥ 0`). The intersection of half-planes is convex. To compute it:

1. Add four half-planes forming a huge box so that the answer is bounded (unbounded regions become large polygons; if the problem cares, check whether a box edge survives).
2. Sort half-planes by the angle of d.
3. Process them in order with a deque of the half-planes that currently define the boundary. When a new half-plane h arrives, the vertex formed by the last two deque lines may now be outside h: pop from the back while that holds. Symmetrically, the angular order wraps around, so the vertex formed by the first two may be outside: pop from the front while that holds.
4. Parallel half-planes with the same direction: keep only the more restrictive one. Opposite directions with no further lines between them: the intersection is empty or degenerate (the cp-algorithms version returns empty).
5. After all insertions, the front of the deque may cut off vertices at the back and vice versa; trim both ends once more.
6. If fewer than 3 half-planes remain the intersection is empty (or degenerate); otherwise consecutive intersections give the polygon.

Why it works. Sorted by angle, the boundary of the intersection uses half-planes in exactly this angular order, each at most once. A half-plane popped from the back is redundant: the new half-plane plus the previous one already cut away everything the popped one contributed. Because each half-plane is pushed and popped at most once, the deque phase is O(n) and the total is O(n log n) for sorting.

Applications.

- *Polygon kernel*: the set of points that see the entire polygon is the intersection of the half-planes of its edges (POJ 3335, POJ 1279).
- *Largest inscribed circle of a convex polygon*: binary search the radius r; move every edge inward by r along its normal; the circle fits iff the shifted half-planes have a non-empty intersection (POJ 3525). Shifting a half-plane is just `p += normal * r`.
- *Two-variable linear programming*: constraints `a x + b y ≤ c` are half-planes; the feasible region is the HPI polygon and a linear objective is optimized at one of its vertices.
- *Voronoi cell of one site*: the intersection of the bisector half-planes against all other sites.

Use `long double` for HPI: intersection points are rational with large denominators, and ε-comparisons on the `out` test decide which lines survive.

4. Closest pair of points

Sweep line. Sort points by x. Keep the best squared distance `best` and let `d = [√best]`. Maintain the set of already processed points whose x lies in `[x − d, x]`, ordered by y. For the new point (x, y), any point closer than d must be in that set with y-coordinate in `[y − d, y + d]`; check exactly those, then insert the new point.

Why only O(1) checks. The candidates lie in a d × 2d rectangle, and any two of them are at distance ≥ √best (otherwise `best` would already be smaller). Divide the rectangle into 8 squares of side d/2: each square has diameter d/√2 < d, so it holds at most one point (up to the rounding of d, which only adds a constant). Hence the inner loop runs O(1) times per point and the whole algorithm is O(n log n) with a balanced BST. Each point is inserted and erased once.

Divide and conquer (Shamos–Hoey) is the textbook alternative: split by the median x, recurse, then check the strip of width 2δ around the split line in y order, where each point needs comparing only to the next ≤ 7 points. Merging by y during the recursion keeps it O(n log n). A **randomized grid** algorithm runs in expected O(n): insert points in random order into a hash grid of cell size equal to the current best distance and rebuild whenever the best distance shrinks (it does so with probability ≤ 2/i at step i).

Variants: bichromatic closest pair (compare only across colors), all pairs within distance r (grid of cell size r, compare to 9 neighbouring cells).

5. Angular sweep

5.1 Exact angular order

Sorting by `atan2` is tempting, but two parallel integer vectors can get different floating values, and nearly parallel vectors can be misordered. The exact comparator splits the plane into the upper half `[0, π)` and the lower half `[π, 2π)`; within a half, a precedes b iff `cross(a, b) > 0`. This is a strict weak ordering for nonzero vectors, and vectors in the same direction compare equal (decide separately how to treat them).

5.2 Two pointers around a center

Typical statement: count subsets of points with some property relative to a center c, e.g., triangles strictly containing c. Translate so c is the origin and sort. A triangle (with no vertex collinear with c through another) fails to contain the origin iff all three vertices lie in an open half-plane through the origin. In that case exactly one of the vertices is the "first" in counter-clockwise order, with the other two inside the half-turn `(θ_i, θ_i + π)`. If `k_i` points lie in that half-turn, vertex i is first for exactly `C(k_i, 2)` bad triangles, so

#containing = C(n, 3) − Σ_i C(k_i, 2).

The `k_i` come from a two-pointer pass over the sorted points, indexing modulo n to wrap around. **CF 1284E *New Year and Castle Construction*** asks for the number of 4-point subsets whose convex hull strictly contains a fifth point, summed over all choices of that fifth point; the same counting with C(k_i, 3) around every center gives O(n² log n).

5.3 Events on a rotating ray

Maximum number of points covered by a circle of radius r. Some optimal circle has a point p on its boundary (translate an optimal circle until a point touches the boundary). Fix p; the center lies on the circle of radius r around p, parameterised by an angle. Another point q with |pq| ≤ 2r is covered for center angles in `[φ − α, φ + α]`, where φ is the direction of q − p and `α = acos(|pq| / 2r)`. Sort the 2(n−1) endpoint events, process starts before ends at equal angles, and track the running count: O(n log n) per p, O(n² log n) total (POJ 1981). Arcs crossing angle ±π can be handled by starting the counter with the arcs that contain the starting angle.

The same pattern — map each object to an angular interval and sweep — answers visibility questions, "best line through a fixed point", and "maximum points in a sector".

6. Reference implementations

All programs compile with `g++ -std=c++17 -O2 -Wall`. They were tested against brute force on hundreds of random inputs: calipers against O(n²) pairwise diameter and an O(n³) width check, HPI against exact rational Sutherland–Hodgman clipping (`fractions.Fraction`), closest pair against all pairs (including coordinates up to 10⁹), and the triangle count against O(n³) enumeration.

6.1 Convex hull, diameter and width

Input: n, then n integer points. Output: number of hull vertices, squared diameter, width.

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

struct P {
    ll x, y;
    P operator-(const P& o) const { return {x - o.x, y - o.y}; }
    bool operator<(const P& o) const { return tie(x, y) < tie(o.x, o.y); }
    bool operator==(const P& o) const { return x == o.x && y == o.y; }
};
ll cross(P a, P b) { return a.x * b.y - a.y * b.x; }
ll cross(P o, P a, P b) { return cross(a - o, b - o); }
```

```cpp
ll dist2(P a, P b) { P d = a - b; return d.x * d.x + d.y * d.y; }

// Andrew's monotone chain; returns hull in counter-clockwise order, no collinear points.
vector<P> convexHull(vector<P> pts) {
    sort(pts.begin(), pts.end());
    pts.erase(unique(pts.begin(), pts.end()), pts.end());
    int n = pts.size();
    if (n < 3) return pts;
    vector<P> h(2 * n);
    int k = 0;
    for (int i = 0; i < n; i++) {                       // lower hull
        while (k >= 2 && cross(h[k - 2], h[k - 1], pts[i]) <= 0) k--;
        h[k++] = pts[i];
    }
    for (int i = n - 2, t = k + 1; i >= 0; i--) {       // upper hull
        while (k >= t && cross(h[k - 2], h[k - 1], pts[i]) <= 0) k--;
        h[k++] = pts[i];
    }
    h.resize(k - 1);
    return h;
}

// Rotating calipers: squared diameter of a convex polygon (CCW, no collinear points).
ll diameter2(const vector<P>& h) {
    int n = h.size();
    if (n == 1) return 0;
    if (n == 2) return dist2(h[0], h[1]);
    ll best = 0;
    for (int i = 0, j = 1; i < n; i++) {
        int ni = (i + 1) % n;
        // advance j while the triangle (h[i], h[ni], h[j]) keeps growing
        while (cross(h[ni] - h[i], h[(j + 1) % n] - h[j]) > 0) j = (j + 1) % n;
        best = max({best, dist2(h[i], h[j]), dist2(h[ni], h[j])});
    }
    return best;
}

// Rotating calipers: minimum width (distance between two parallel support lines).
double minWidth(const vector<P>& h) {
    int n = h.size();
    if (n < 3) return 0;
    double best = 1e18;
    for (int i = 0, j = 1; i < n; i++) {
        int ni = (i + 1) % n;
        while (cross(h[ni] - h[i], h[(j + 1) % n] - h[j]) > 0) j = (j + 1) % n;
        P e = h[ni] - h[i];
        best = min(best, (double)cross(e, h[j] - h[i]) / sqrt((double)dist2(h[i],
h[ni])));
    }
    return best;
}

int main() {
    int n;
    cin >> n;
```

```
    vector<P> pts(n);
    for (auto& p : pts) cin >> p.x >> p.y;
    vector<P> h = convexHull(pts);
    cout << h.size() << " " << diameter2(h) << " " << fixed << setprecision(6) <<
minWidth(h) << "\n";
}
```

6.2 Half-plane intersection

Input: n, then n lines `x1 y1 x2 y2`; each keeps the left side of the directed line. Output: area of the intersection (clipped to a box of half-size 10^9).

```
#include <bits/stdc++.h>
using namespace std;
typedef long double ld;
const ld EPS = 1e-12, INF = 1e9;

struct Pt {
    ld x, y;
    Pt operator+(const Pt& o) const { return {x + o.x, y + o.y}; }
    Pt operator-(const Pt& o) const { return {x - o.x, y - o.y}; }
    Pt operator*(ld k) const { return {x * k, y * k}; }
};
ld cross(Pt a, Pt b) { return a.x * b.y - a.y * b.x; }
ld dot(Pt a, Pt b) { return a.x * b.x + a.y * b.y; }

// Half-plane: the points to the LEFT of the directed line p -> p + d.
struct HP {
    Pt p, d; ld ang;
    HP(Pt a, Pt b) : p(a), d(b - a), ang(atan2l(d.y, d.x)) {}
    bool out(Pt r) const { return cross(d, r - p) < -EPS; }   // strictly outside
};
Pt inter(const HP& s, const HP& t) {
    ld a = cross(t.p - s.p, t.d) / cross(s.d, t.d);
    return s.p + s.d * a;
}

// Returns the vertices (CCW) of the intersection; empty if it is empty or degenerate.
// A big bounding box is added so the result is always bounded.
vector<Pt> halfplaneIntersection(vector<HP> H) {
    Pt box[4] = {{INF, INF}, {-INF, INF}, {-INF, -INF}, {INF, -INF}};
    for (int i = 0; i < 4; i++) H.push_back(HP(box[i], box[(i + 1) % 4]));
    sort(H.begin(), H.end(), [](const HP& a, const HP& b) { return a.ang < b.ang; });
    deque<HP> dq;
    for (auto& h : H) {
        while (dq.size() > 1 && h.out(inter(dq[dq.size() - 1], dq[dq.size() - 2])))
dq.pop_back();
        while (dq.size() > 1 && h.out(inter(dq[0], dq[1]))) dq.pop_front();
        if (!dq.empty() && fabsl(cross(h.d, dq.back().d)) <= EPS) {
            if (dot(h.d, dq.back().d) < 0) return {};          // opposite, parallel
            if (h.out(dq.back().p)) dq.pop_back();             // same direction: keep
the
```

```
            else continue;                                      // more restrictive one
        }
        dq.push_back(h);
    }
    while (dq.size() > 2 && dq[0].out(inter(dq[dq.size() - 1], dq[dq.size() - 2])))
dq.pop_back();
    while (dq.size() > 2 && dq.back().out(inter(dq[0], dq[1]))) dq.pop_front();
    if (dq.size() < 3) return {};
    vector<Pt> poly;
    for (size_t i = 0; i < dq.size(); i++) poly.push_back(inter(dq[i], dq[(i + 1) %
dq.size()]));
    return poly;
}

ld area(const vector<Pt>& poly) {
    ld s = 0;
    for (size_t i = 0; i < poly.size(); i++) s += cross(poly[i], poly[(i + 1) %
poly.size()]);
    return s / 2;
}

int main() {
    int n;
    cin >> n;
    vector<HP> H;
    for (int i = 0; i < n; i++) {
        ld x1, y1, x2, y2;
        cin >> x1 >> y1 >> x2 >> y2;     // keep the left side of (x1,y1) -> (x2,y2)
        H.push_back(HP({x1, y1}, {x2, y2}));
    }
    cout << fixed << setprecision(6) << area(halfplaneIntersection(H)) << "\n";
}
```

6.3 Closest pair (sweep line)

Input: n ≥ 2, then n integer points (|coordinates| ≤ 10^9). Output: minimum squared distance — exactly the output format of CSES 2194. Runs in under 0.2 s for n = $2 \cdot 10^5$.

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

// Minimum squared Euclidean distance among n >= 2 points (sweep line, O(n log n)).
ll closestPair(vector<pair<ll, ll>> pts) {
    sort(pts.begin(), pts.end());                           // by x, then y
    multiset<pair<ll, ll>> active;                          // (y, x) of points with x in
window
    ll best = LLONG_MAX;
    size_t left = 0;
    for (auto& [x, y] : pts) {
        ll d = (ll)ceil(sqrt((long double)best));           // current distance bound
        while (left < pts.size() && pts[left].first < x - d) {   // drop points too far
```

```
left
            active.erase(active.find({pts[left].second, pts[left].first}));
            left++;
        }
        for (auto it = active.lower_bound({y - d, LLONG_MIN}); it != active.end() && it-
>first <= y + d; ++it) {
            ll dx = x - it->second, dy = y - it->first;
            best = min(best, dx * dx + dy * dy);
        }
        active.insert({y, x});
    }
    return best;
}

int main() {
    int n;
    cin >> n;
    vector<pair<ll, ll>> pts(n);
    for (auto& [x, y] : pts) cin >> x >> y;
    cout << closestPair(pts) << "\n";
}
```

6.4 Angular sweep: triangles containing the origin

Input: n, then n integer points, none at the origin and no two collinear with the origin. Output: number of triangles strictly containing the origin.

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

struct P { ll x, y; };
ll cross(P a, P b) { return a.x * b.y - a.y * b.x; }
int half(P a) { return (a.y < 0 || (a.y == 0 && a.x < 0)) ? 1 : 0; }   // [0, pi) -> 0
bool angleLess(P a, P b) {                                             // exact, no atan2
    if (half(a) != half(b)) return half(a) < half(b);
    return cross(a, b) > 0;
}

// Number of triangles with vertices among pts that strictly contain the origin.
// Assumes no point is the origin and no two points are collinear with the origin.
ll trianglesContainingOrigin(vector<P> pts) {
    int n = pts.size();
    sort(pts.begin(), pts.end(), angleLess);
    ll total = (ll)n * (n - 1) * (n - 2) / 6, bad = 0;
    // for each i, count points strictly within the open half-turn (theta_i, theta_i +
pi)
    for (int i = 0, j = 0; i < n; i++) {
        if (j < i + 1) j = i + 1;
        while (j < i + n && cross(pts[i], pts[j % n]) > 0) j++;
        ll k = j - i - 1;                       // points in the half-turn after pts[i]
        bad += k * (k - 1) / 2;                 // triangle lies in a half-plane through
```

```
0
    }
    return total - bad;
}

int main() {
    int n;
    cin >> n;
    vector<P> pts(n);
    for (auto& p : pts) cin >> p.x >> p.y;
    cout << trianglesContainingOrigin(pts) << "\n";
}
```

7. Common mistakes

- **Collinear or duplicate hull points** fed to calipers. Build the hull with `<= 0` pops and deduplicate first.
- **Degenerate hulls.** One point, two points, or all points collinear (the hull has two vertices). Handle before the main loop.
- **Overflow.** With coordinates up to 10^9 a cross product of differences is up to $8 \cdot 10^{18}$ — inside `long long`, but any product of two cross products, or a squared distance of coordinates up to $2 \cdot 10^9$, needs `__int128`.
- **HPI conventions.** Mixing "left side" and "right side", forgetting the bounding box (so the deque never closes), or using `double` and an ε that is too large/small for the coordinate scale. Shifted half-planes in a binary search must move *inward*.
- **HPI degenerate answers.** When the intersection is a single point or segment, the algorithm may report empty or a zero-area polygon. Read whether the problem treats "touching" as feasible and, if so, binary search with a tiny tolerance.
- **Closest pair window rounding.** Using $\lfloor\sqrt{best}\rfloor$ can exclude a point at exactly the boundary; round **up**. Use a `multiset` (or include x in the key) so duplicate points are not lost.
- **Angle sorting with `atan2`**, especially for problems that ask about parallel or collinear configurations. Use the half + cross comparator.
- **Wrap-around in angular two pointers.** The window may run past the end of the sorted array; index modulo n and cap the window at $n - 1$ points.

8. Practice problems

Calipers and hull:

- CSES 2195 — Convex Hull — https://cses.fi/problemset/task/2195
- POJ 2187 — Beauty Contest — http://poj.org/problem?id=2187
- Kattis — Robert Hood — https://open.kattis.com/problems/roberthood

- UVa 10173 — Smallest Bounding Rectangle — UVa Online Judge, https://onlinejudge.org/ (search by number)
- Codeforces 1142C — U2 — https://codeforces.com/problemset/problem/1142/C

Half-plane intersection:

- POJ 3335 — Rotating Scoreboard — http://poj.org/problem?id=3335
- POJ 1279 — Art Gallery — http://poj.org/problem?id=1279
- POJ 3525 — Most Distant Point from the Sea — http://poj.org/problem?id=3525

Closest pair:

- CSES 2194 — Minimum Euclidean Distance — https://cses.fi/problemset/task/2194
- Kattis — closestpair2 — https://open.kattis.com/problems/closestpair2

Angular sweep:

- Library Checker — Sort Points by Argument — https://judge.yosupo.jp/problem/sort_points_by_argument
- Codeforces 598C — Nearest vectors — https://codeforces.com/problemset/problem/598/C
- Codeforces 1284E — New Year and Castle Construction — https://codeforces.com/problemset/problem/1284/E
- POJ 1981 — Circle and Points — http://poj.org/problem?id=1981
- Codeforces 1025F — Disjoint Triangles (stretch) — https://codeforces.com/problemset/problem/1025/F

9. Further reading

- M. de Berg, O. Cheong, M. van Kreveld, M. Overmars, *Computational Geometry: Algorithms and Applications* — chapters on convex hulls, half-plane intersection / linear programming, and Voronoi diagrams.
- G. Toussaint, "Solving geometric problems with the rotating calipers", Proc. IEEE MELECON, 1983.
- H. Freeman, R. Shapira, "Determining the minimum-area encasing rectangle for an arbitrary closed curve", CACM, 1975.
- cp-algorithms: "Half-plane intersection – S&I algorithm", "Finding the nearest pair of points" — https://cp-algorithms.com/
- KACTL geometry section (`ConvexHull.h`, `HullDiameter.h`, `ClosestPair.h`) — https://github.com/kth-competitive-programming/kactl
- World Finals track, Lecture 6 continues with 3D hulls, Delaunay/Voronoi, Minkowski sums and exact arithmetic.